

setup-gcp

Retargeting the repo at your own GCP account / another laptop

`runbook-gcp.md` is the provisioning sequence. This file is the other half: what is environment-specific, what you must edit, and what genuinely differs on a fresh account.

Nothing in the repo is tied to the project it was developed against. Everything environment-specific lives in exactly three places:

What	Where	In git?
Project id, region, billing account	<code>local/scripts/gcp/_env.sh</code> (read from your shell)	no — values never committed
Service-account key	<code>terraform/envs/dev/sa-json-keys/*.json</code>	no — directory is gitignored
Local runtime config	<code>.env</code> (copied from <code>.env.example</code>)	no

A hardcoded project id anywhere else is a bug — fix it rather than working around it. `mig-local` appears as a *default* in a few places for the emulator stack; it is never a real project.

1. What you need first

In GCP:

- A project you own, or permission to create one.
- An **OPEN** billing account linked to it (`gcloud billing accounts list`; if `OPEN: False`, nothing will provision).
- A service account with `roles/owner` for the initial bootstrap, or the narrower set in `terraform/modules/iam/main.tf` if you prefer to pre-create resources.

On your laptop. The GCP path does *not* need the local emulator stack, but it is not tooling-free either: two artefact types are built on your machine and pushed rather than built by a cloud builder.

Tool	Why the GCP path needs it
<code>gcloud</code> , <code>terraform</code>	Everything.
docker or podman	The images the Composer DAG runs as pods are built locally and pushed — there is no <code>gcloud builds submit</code> path wired in. <code>build_java_images.sh</code> and <code>make build-templates</code> detect <code>podman</code> , else <code>docker</code> , else hard-fail.
JDK 25 + Maven	<code>make java-build</code> produces the jars baked into those images.
<code>gpg</code>	<code>make seed-secrets</code> uploads <code>local/keys/seckey.asc</code> to Secret Manager and refuses to run without it. Generate it with <code>make keys</code> .

No host Python. Every GCP target is `bash + gcloud + terraform + a container build`. The Python runs *remotely* — Beam pipelines inside pod images on GKE, the DAG on Composer. That includes `make smoke-gcp` , which builds `Dockerfile.toolbox` (Python 3.11 + Beam + JRE 25 + the app jars) and runs the orchestrator inside it, mounting your ADC key and `local/keys` .

Note the contrast with the local stack, where **podman is mandatory** (`make up` drives `podman-compose` and nothing else). Here the two runtimes are interchangeable — these images are pushed to a registry or run once, so nothing depends on rootless podman's behaviour.

2. Point the repo at your project

One edit is unavoidable. The Terraform backend bucket is hardcoded at `terraform/envs/dev/main.tf:31` (`bucket = "mig-000001-1-dev-tfstate"`), because a backend block cannot use variables. Change it to your own `<project>-tfstate` before the first `init` . This is the only file you must edit to retarget the repo.

```
# Your values – none of these are committed anywhere.
export TF_VAR_project_id=my-project-dev
export TF_VAR_billing_account=XXXXXX-XXXXXX-XXXXXX
export TF_VAR_region=europe-west1           # optional, this is the default

# Drop the SA key here; the directory is gitignored and _env.sh finds it automatically
# when exactly one .json file is present.
mkdir -p terraform/envs/dev/sa-json-keys
cp ~/Downloads/my-sa-key.json terraform/envs/dev/sa-json-keys/

source local/scripts/gcp/_env.sh           # validates the above and exports ADC
```

`_env.sh` fails loudly with the exact missing variable rather than half-configuring, so if it is silent you are ready. The `make` targets that talk to GCP (`build-templates` , `deploy-dags` , `deploy-dataform` , `seed-secrets`) all read the environment through it.

Because the project `mig-000001-1-dev` was created manually — a service account cannot create a project under "No organization" — Terraform must **adopt** it rather than recreate it. Pass `-var="create_project=false"` on every command (`terraform.tfvars` is not gitignored, so the override is passed inline rather than committed).

Then provision and deploy per [runbook-gcp.md](#) §B.

3. Things that will differ on your account

Honest list, from doing this against a second project:

- **APIs enable slowly.** The first apply may fail on a service not yet active. Re-running `terraform apply` usually resolves it — enabling is asynchronous.
- **Artifact Registry needs an explicit login** before the first push: `gcloud auth configure-docker ${TF_VAR_region}-docker.pkg.dev` (podman users may need `podman login` with an ADC token).
- **Managed Kafka's bootstrap is VPC-only.** It resolves to NXDOMAIN from a laptop. Use `--sinks gcs` unless you are running inside the VPC.
- **Composer's Cloud SQL instance is invisible** — `gcloud sql instances list` returns nothing, because it lives in a Google-managed tenant project. Not a stall.
- **Quotas.** A fresh project has low Dataflow CPU quota; three concurrent pipelines can hit it. Request an increase or run the stages serially.

4. Moving an existing deployment to a new laptop

The Terraform state lives in a **GCS bucket**, not on disk, so there is nothing to copy:

```
export TF_VAR_project_id=... TF_VAR_billing_account=...
cp <sa-key>.json terraform/envs/dev/sa-json-keys/
source local/scripts/gcp/_env.sh
terraform -chdir=terraform/envs/dev init      # re-attaches to the remote state
terraform -chdir=terraform/envs/dev plan      # should report "No changes"
```

A `plan` reporting **No changes** confirms the new machine is correctly attached to the existing environment. If it proposes to create things that already exist, the backend bucket at `terraform/envs/dev/main.tf:31` is not the one holding your state — fix that before applying anything, or you will provision a second copy of the environment.